

Trabalho DSID

Integrantes do Grupo:

15456938 - Guilherme Hiyuki Maruta

13828007 – Guilherme Padun Santos

15674466 – Gustavo Henrique Ferreira Alves

15463492 – João Pedro Nunes Aquino

15639956 – Marcos Yutoh Mori

Introdução

Este documento tem como intuito apresentar a especificação de um sistema distribuído capaz de administrar o rastreo de um tipo de usuário (entregadores) por outro tipo de usuário (cliente). O foco principal desse sistema é lidar com possíveis falhas de queda de forma que o cliente sinta/note o mínimo possível estas falhas até que essa falha seja contornada.

Arquitetura

Modelo

Modelo Físico: Cliente-Servidor em nuvem

O aplicativo no celular do entregador, o celular do cliente e o aplicativo do restaurante atuam como clientes do sistema. Esses clientes não se comunicam diretamente entre si; todas as interações passam por um servidor central, hospedado em uma infraestrutura de computação em nuvem (como AWS, Google Cloud ou Azure), responsável por processar as requisições e coordenar o funcionamento do sistema.

Modelo Lógico: Publish-Subscribe ou Event-Based

Para permitir a atualização em tempo real da localização do entregador no mapa do cliente, o sistema pode utilizar o modelo Publish-Subscribe. Nesse modelo, o aplicativo do entregador publica sua localização GPS no sistema, enquanto o Servidor Rastreador responsável pelo pedido consome essas atualizações, mantém o estado mais recente do rastreamento e encaminha as informações ao cliente. Dessa forma, os componentes permanecem desacoplados, pois o entregador não precisa conhecer diretamente o cliente, nem vice-versa.

Para pedidos ainda sem entregador correspondente, é necessário um PubSub, a partir daí os entregadores que estariam subscritos podem solicitar que sejam responsáveis por um pedido específico.

Software interno

A arquitetura interna do sistema pode seguir o modelo de arquitetura em camadas (multitier), que divide a aplicação em três níveis principais:

Camada de Apresentação (Interface do Usuário)

Responsável pela interação com o usuário. Inclui o aplicativo do cliente, do entregador e do restaurante. Essa camada exibe informações como o mapa, o status do pedido e as notificações, sem realizar processamento complexo.

Simulação de entregadores:

Para viabilizar testes no contexto da disciplina, os entregadores não serão usuários reais, mas sim simulados através de scripts automatizados.

Esses scripts enviarão periodicamente atualizações de localização (latitude, longitude, timestamp), permitindo reproduzir cenários de múltiplos entregadores ativos simultaneamente.

Isso possibilita a realização de testes de carga e comportamento do sistema sem dependência de dispositivos móveis reais.

Aplicação de restaurantes:

Para simplificação da prova de conceito, os restaurantes serão considerados entidades pré-cadastradas (hardcoded) no sistema.

Dessa forma, reduz-se a complexidade da implementação e permite maior foco nos aspectos distribuídos do sistema.

Interface de acesso:

A prova de conceito será implementada utilizando aplicações simples (por exemplo, aplicações de console ou web), eliminando a necessidade de desenvolvimento mobile.

Essa decisão permite focar nos aspectos de comunicação distribuída, tolerância a falhas e escalabilidade.

Camada de Lógica de Negócios (Processamento)

Executada nos servidores na nuvem, essa camada contém as regras de negócio do sistema. É responsável por funções como atribuir entregadores aos pedidos, calcular tempo estimado de entrega e processar pagamentos. Em arquiteturas modernas, essa camada pode ser implementada utilizando microsserviços, nos quais cada serviço é responsável por uma funcionalidade específica do sistema.

Camada de Dados

Responsável pelo armazenamento persistente das informações do sistema. Inclui bancos de dados que guardam registros de usuários, pedidos, restaurantes, histórico de entregas e informações de pagamento.

Middleware

Middleware será utilizado nesse sistema distribuído, pois ele facilita a comunicação entre diferentes componentes da aplicação.

Para o rastreamento em tempo real, será usado um Message Broker (corretor de mensagens), que é um tipo de Middleware Orientado a Mensagens (MOM). Nesse modelo, os aplicativos enviam mensagens para um intermediário responsável por distribuí-las para os interessados.

Uma analogia útil é pensar no Message Broker como uma central de triagem dos Correios. O entregador envia a informação de localização ao sistema, e o broker se encarrega de encaminhá-la ao Servidor Rastreador responsável pelo pedido, que então atualiza o estado do rastreamento e repassa a informação aos clientes interessados.

Testes do sistema

Testes de Resiliência de Rede

Como dispositivos móveis podem perder sinal com frequência, os testes vão simular situações em que o entregador entra em áreas sem cobertura de rede. O sistema do lado dos entregadores deve garantir que os dados de localização sejam armazenados temporariamente no dispositivo e enviados quando a conexão for restabelecida.

Testes de Integração Fim a Fim (End-to-End)

Esse tipo de teste valida o funcionamento completo do sistema. Um usuário realiza um pedido, o restaurante o aceita e o entregador realiza a entrega, permitindo verificar se todas as partes do sistema (cliente, restaurante e entregador) estão corretamente sincronizadas.

Comunicação

Tipos de Comunicação

A comunicação do sistema é majoritariamente assíncrona, baseada em troca de mensagens via um broker (modelo publish-subscribe).

Para eventos em tempo real (como localização), utiliza-se conexão duradoura (ex: WebSocket ou TCP persistente).

Para operações pontuais (como criação de pedido), utiliza-se comunicação síncrona via requisições curtas (request/response).

Tipos de Mensagens

Mensagens de comando (requisições diretas): request/response (cliente -> servidor)

Mensagem: CriarPedido

```
{  
  "idPedido": "uuid",  
  "idCliente": "123",  
  "idRestaurante": "456",  
  "timestamp": 1710000000  
}
```

Mensagem: AceitarPedido

```
{  
  "idPedido": "uuid",  
  "idEntregador": "789",  
  "timestamp": 1710000001  
}
```

```
}  
Mensagem: ConfirmarEntrega  
{  
  "idPedido": "uuid",  
  "idCliente": "123",  
  "timestamp": 1710000003  
}
```

Mensagens de evento (modelo Publish-Subscribe)

Mensagem: PedidoDisponivel (PUBLISH)

```
{  
  "idPedido": "uuid",  
  "idRestaurante": "456",  
  "timestamp": 1710000000  
}
```

Mensagem: LocalizacaoEntregador (PUBLISH)

```
{  
  "idEntregador": "789",  
  "idPedido": "uuid",  
  "latitude": -23.55,  
  "longitude": -46.63,  
  "timestamp": 1710000002  
}
```

Mensagem: EventoLocalizacao (ENTREGUE PELO BROKER AO SERVIDOR RASTREADOR)

```
{  
  "idPedido": "uuid",  
  "latitude": -23.55,  
  "longitude": -46.63,  
  "timestamp": 1710000002  
}
```

Mensagem: SubscribeRastreo

```
{  
  "idPedido": "uuid",  
  "idServidorRastreador": "R1"  
}
```

Mensagens internas (infraestrutura do sistema)

keepAlive (Servidor -> Servidor)

```
{  
  "idServidor": "R1",  
  "tipoServidor": "RASTREADOR",  
  "timestamp": 1710000004  
}
```

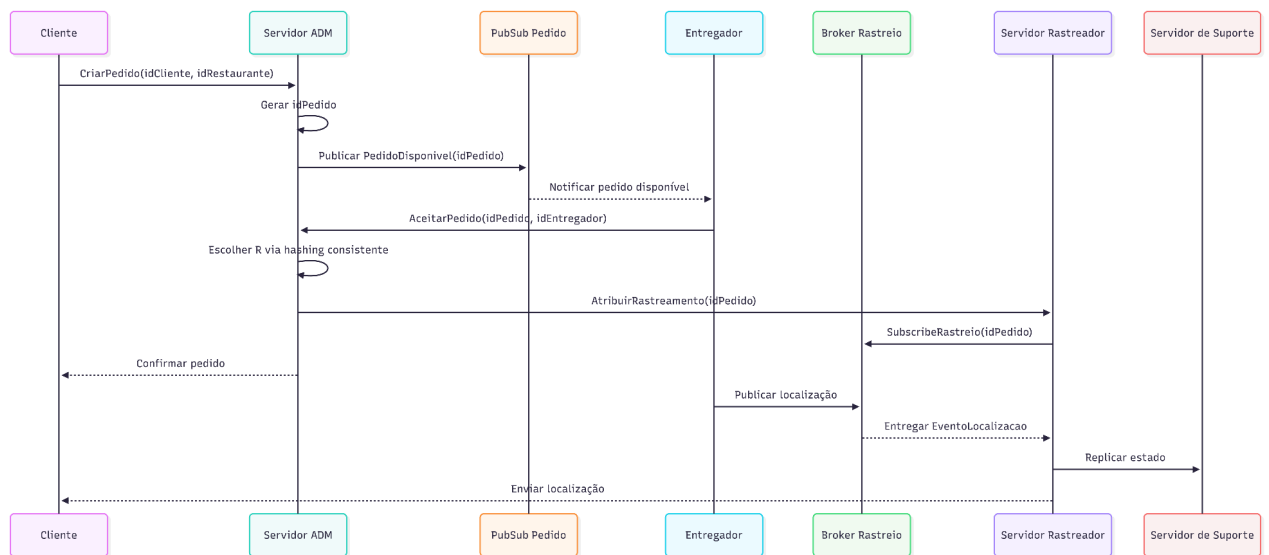
- Verificar disponibilidade entre nós por meio de heartbeats periódicos

AtualizacaoRoteamento (Servidor -> Servidor)

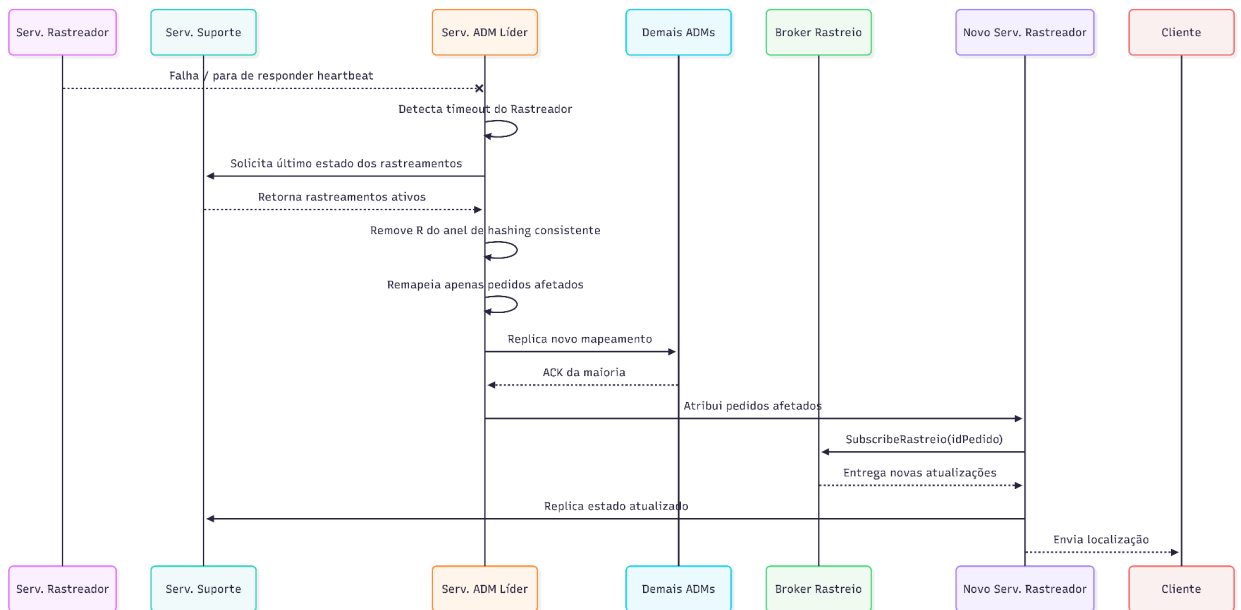
- Atualiza qual servidor está responsável por quais pedidos

Diagramas de seqüências

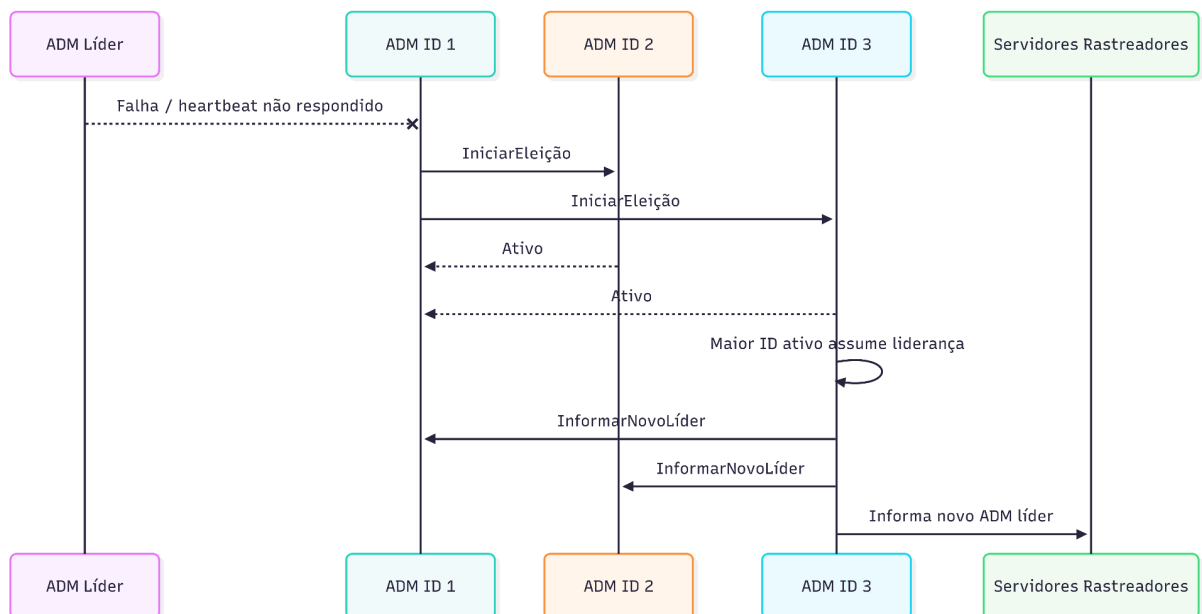
Fluxo de Criação de um pedido



Queda de componente do sistema

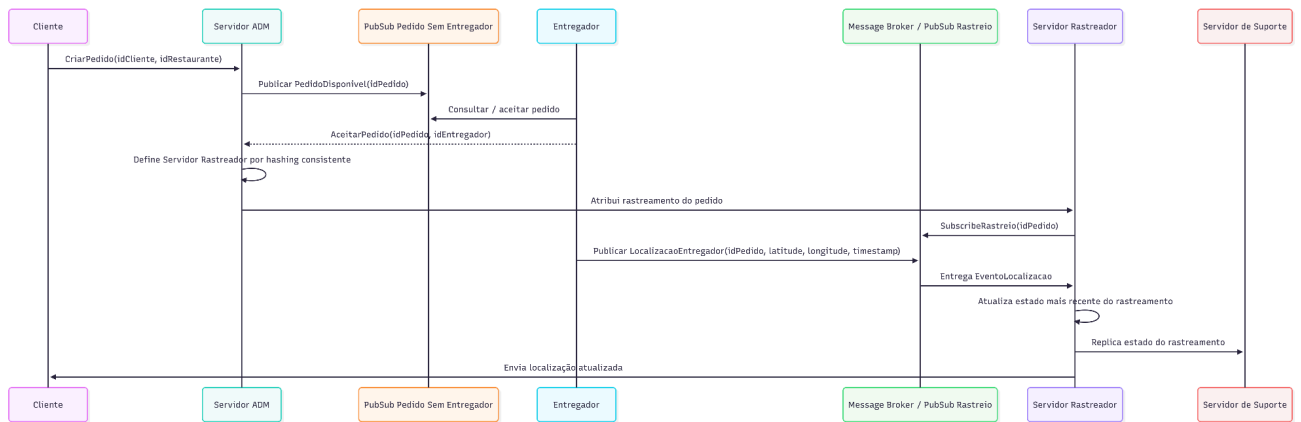


Queda do Servidor ADM líder



*No caso de reeleição, o cliente pode enviar a requisição para qualquer servidor ADM. Se esse servidor não for o líder, ele responde, informando qual é o servidor ADM líder atual, o cliente ao receber essa informação reenvia a criação de pedido.

Tarefa dos componentes do sistema



Nomeação

Garantia de unicidade

Para garantir a unicidade global dos identificadores (principalmente de pedidos), será utilizado um identificador único gerado no momento da criação do pedido.

Esse identificador pode ser implementado como um UUID (Universally Unique Identifier) ou como uma combinação de:

- ID do cliente
- Timestamp de alta precisão
- Um contador incremental local

Dessa forma, mesmo em um ambiente distribuído com múltiplos servidores, é possível garantir que não haverá colisão de identificadores.

Determinação e Resolução de nome dos servidores

Para determinar qual Servidor Rastreador é responsável por um determinado pedido, será utilizado um mecanismo determinístico baseado em hashing consistente (Consistent Hashing).

Nesse modelo, os Servidores Rastreadores são organizados em um anel lógico de hashes. Cada Servidor Rastreador possui um identificador único e é representado no anel por nós virtuais, calculados a partir de $\text{hash}(\text{idServidor} + \text{"\#"} + \text{indice})$. O uso de nós virtuais melhora o balanceamento da carga, pois permite que um mesmo servidor seja distribuído por diferentes posições do anel.

Quando um pedido é criado, calcula-se $\text{hash}(\text{idPedido})$ para obter sua posição no anel. O Servidor Rastreador responsável será o primeiro nó virtual encontrado no sentido horário cuja posição seja maior ou igual ao hash do pedido. Caso a busca chegue ao fim do anel, ela retorna ao início. O servidor real associado a esse nó virtual passa a ser responsável pelo rastreamento do pedido.

Dessa forma, qualquer componente que conheça a lista de Servidores Rastreadores ativos e a configuração do anel consegue determinar qual servidor é responsável por um pedido. Além disso, quando um servidor entra ou sai do sistema, apenas os pedidos pertencentes aos intervalos afetados do anel precisam ser remapeados, evitando a redistribuição completa que ocorreria com uma estratégia simples como $\text{hash}(\text{idPedido}) \bmod N$.

Recursos nomeados/identificados

Cliente e seu conjunto de pedidos

Entregador

Restaurante

Pedido Recém Criado

Pedido com Entregador

Pedido com Servidor Rastreador Atribuído

Servidor Administrador (porque se termos vários, precisamos ter um método de identificação porque é necessário realizar eleição de líder ou reeleição)

Servidor Rastreador

Servidor de Suporte

Esquema de nomeação

Baseado em atributos:

- Cliente: ID como identificador, se assumirmos que o Cliente é incapaz de se mover, seu IP também pode ser um meio de identificação mais conveniente
<ID/IP>
- Entregador: Id puramente, porque seu IP pode se alterar visto que ele vai se mover
<ID>

- Restaurante: Só ID, e podem ser hardcoded (se não forem podem ser por IP porque restaurantes são fixos)
<ID/IP>
- Pedido Recém Criado
<id_cliente/IP_cliente, id_Restaurante, timestap>
- Pedido com Entregador
<id_cliente/IP_cliente, id_Entregador, id_Restaurante, timestamp>
Timestamp será de quando ele foi criado
- Pedido com Servidor Rastreador Atribuído
<id_cliente/IP_cliente, id_Entregador, id_Restaurante, timestamp, servidorRastreioResponsável>
Ter a informação do responsável aqui possibilita que um cliente possa perguntar diretamente a localização para o servidor de rastreo, essa informação porém pode mudar se um servidor de rastreo cair então o último campo deverá ser mútavel.
- Servidor Rastreador
<ID/IP>
- Servidor de administração
<ID/IP>
- Servidor de Suporte
<ID/IP , ID_Servidor_de_Rastreio>
Já que cada servidor de rastreo tem um de suporte associado

Paralelismo

Gerenciamento de Estado

O sistema adotará uma abordagem híbrida em relação ao gerenciamento de estado. Do ponto de vista dos clientes, entregadores e restaurantes, o sistema será stateless, pois não dependerá de sessões persistentes nem do histórico completo de pedidos ou rastreamentos para processar novas requisições. No entanto, os servidores manterão o estado operacional necessário ao funcionamento da aplicação, como os rastreamentos ativos, o mapeamento entre pedidos e Servidores Rastreadores e as réplicas utilizadas para recuperação de falhas. Dessa forma, diferentes servidores podem assumir responsabilidades após falhas sem perda significativa de informações, desde que o estado relevante tenha sido replicado corretamente.

Threads

Threads serão utilizadas com o intuito de diminuir o impacto de uma falha a partir da perspectiva de um usuário, ao possuir uma thread separadamente para renderizar a interface, no caso de falha de rede é possível permanecer mostrando informações ao usuário.

Cliente:

- Thread de networking (recebe localização)
- Thread do cálculo do tempo de entrega
- Thread de renderização de localização e interface

Entregador, Restaurante, Servidor Administrador, Servidor de Rastreo, Servidor de Suporte:

- Thread de networking

Virtualização

Virtualização não será utilizada em nenhum servidor, não foi vista a necessidade. Da perspectiva dos clientes e entregadores foi considerado para possibilitar a execução do programa.

Coordenação

Sincronização

O sistema necessita de um mecanismo de sincronização para garantir que as atualizações de localização dos entregadores sejam processadas na ordem correta. Para isso, cada atualização enviada pelo aplicativo do entregador contém um timestamp, que permite identificar a versão mais recente da informação. No servidor e nos clientes, caso múltiplas atualizações sejam recebidas fora de ordem devido à latência da rede, apenas a mensagem com o timestamp mais recente será considerada válida e utilizada para atualizar a posição exibida no mapa. Dessa forma, evita-se que localizações antigas sobrescrevam informações mais recentes, mantendo a consistência do rastreamento em tempo real.

Exclusão Mútua

Não será necessário empregar mecanismos de exclusão mútua distribuída, pois o sistema não possui recursos compartilhados que exijam acesso exclusivo por diferentes servidores. As principais operações consistem no envio e no processamento de atualizações de localização dos entregadores, que são independentes entre si. Caso duas atualizações referentes ao mesmo entregador sejam recebidas em instantes próximos, a mensagem com o timestamp mais recente simplesmente substitui a anterior, sem gerar inconsistências. Além disso, a arquitetura baseada em Publish-Subscribe e em comunicação assíncrona reduz significativamente a necessidade de controle rígido de concorrência, uma vez que os componentes do sistema operam de forma desacoplada.

Algoritmo de Eleição

Será utilizado um algoritmo de eleição para garantir a disponibilidade do sistema em caso de falha do servidor responsável pela coordenação. A arquitetura prevê a existência de múltiplos servidores administrativos, sendo que apenas um atua como líder em determinado momento. Caso esse servidor deixe de responder, os demais detectam sua indisponibilidade e iniciam um processo de eleição para definir um novo líder.

A opção escolhida para esse cenário é o Bully Algorithm, no qual cada servidor possui um identificador único (ID). Quando um servidor detecta a falha do líder, inicia uma eleição enviando mensagens aos servidores com identificadores superiores. Se nenhum deles responder, o próprio servidor assume a liderança; caso contrário, o servidor ativo com o maior ID torna-se o novo líder e comunica essa informação aos demais participantes do sistema. Esse algoritmo foi escolhido por sua simplicidade de implementação, rapidez na recuperação e boa adequação a clusters pequenos, como o considerado neste projeto.

Após a eleição, o novo Servidor Administrador líder passa a coordenar todas as alterações na tabela de mapeamento entre pedidos e Servidores Rastreadores. A propagação dessas alterações para os demais Servidores Administradores é realizada pelo mecanismo de replicação descrito na seção de Protocolos de Consistência, não fazendo parte do processo de eleição.

Utilização de Pub Sub

O modelo Publish-Subscribe será implementado por meio de um Message Broker, responsável por intermediar a comunicação entre os componentes do sistema. O aplicativo do entregador atua como publicador, enviando periodicamente mensagens contendo seu identificador, o identificador do pedido, latitude, longitude e timestamp para o tópico de localização do pedido.

O Servidor Rastreador responsável pelo pedido atua como consumidor deste tópico. Ao receber uma atualização de localização, ele atualiza o estado mais recente do rastreamento, replica essa informação para seu Servidor de Suporte e encaminha a atualização aos clientes e restaurantes interessados. Dessa forma, o Message Broker desacopla o entregador do restante do sistema, enquanto o Servidor Rastreador concentra o gerenciamento do estado e a entrega das informações de rastreamento aos interessados.

Esse modelo elimina a necessidade de comunicação direta entre entregador e clientes, reduzindo o acoplamento entre os componentes e facilitando a escalabilidade do sistema. Além disso, outros componentes internos do sistema podem ser adicionados como consumidores dos tópicos do Message Broker sem necessidade de alterações no aplicativo do entregador.

Fluxo de comunicação:

Entregador → Message Broker → Servidor Rastreador → Cliente / Restaurante

Servidor Rastreador → Servidor de Suporte

Replicação e Consistência

Replicação

O sistema utiliza mecanismos de replicação com o objetivo de aumentar sua disponibilidade e tolerância a falhas. Tanto componentes quanto dados importantes possuem réplicas, permitindo que o funcionamento do sistema seja mantido mesmo diante da indisponibilidade de alguns servidores.

Caso o sistema não utilizasse replicação, haveria pontos únicos de falha (Single Points of Failure). Por exemplo, a indisponibilidade de um servidor responsável pelo rastreamento de uma entrega faria com que o cliente deixasse de receber atualizações da localização do entregador até que o serviço fosse restaurado manualmente. Da mesma forma, a falha do servidor responsável pela coordenação impediria a criação e o gerenciamento de novos pedidos durante o período de indisponibilidade.

Os principais elementos replicados são as listas de rastreamento, os servidores rastreadores e os servidores administradores. As informações de localização dos entregadores são mantidas simultaneamente em, pelo menos, dois servidores. Para cada Servidor Rastreador (R), existe um Servidor de Suporte (SUP), responsável por armazenar uma cópia atualizada dos rastreamentos sob responsabilidade daquele servidor. Além disso, o sistema conta com pelo menos três Servidores Administradores (ADM), que mantêm de forma consistente a relação entre pedidos e os respectivos servidores de rastreamento responsáveis.

A cada atualização de localização recebida pelo Servidor Rastreador, a mesma informação é enviada ao seu Servidor de Suporte. O Servidor de Suporte mantém apenas o estado mais recente de cada pedido sob responsabilidade daquele Rastreador, incluindo o identificador do pedido, o identificador do entregador, a localização e o timestamp da última atualização. Dessa forma, caso o Servidor Rastreador falhe, o Servidor de Suporte possui informações suficientes para auxiliar na recuperação dos rastreamentos ativos.

Nesse modelo, o Servidor Administrador (ADM) não atua como uma cópia completa dos rastreamentos. Sua função principal é coordenar o mapeamento entre pedidos e Servidores Rastreadores, participar da eleição de líder e conduzir a redistribuição de responsabilidades em caso de falha. Já o Servidor de Suporte (SUP) atua como réplica operacional de um Servidor Rastreador específico, permitindo a recuperação dos dados desse rastreador caso ele falhe.

Modelo de Consistência

O projeto adota um modelo de consistência eventual (Eventual Consistency), adequado para aplicações de rastreamento em tempo real, nas quais pequenas inconsistências temporárias são aceitáveis em troca de maior disponibilidade e desempenho.

Cada atualização de localização enviada pelo entregador contém um timestamp, utilizado para determinar a versão mais recente da informação. Assim, caso múltiplas atualizações referentes ao mesmo entregador sejam recebidas em ordem diferente daquela em que foram geradas, apenas a mensagem com o timestamp mais recente será considerada válida, substituindo automaticamente as versões anteriores. Dessa forma, não é necessário utilizar bloqueios ou mecanismos de exclusão mútua distribuída.

Esse modelo admite que, durante situações de falha ou recuperação do sistema, possam existir divergências temporárias entre as réplicas. Por exemplo, um Servidor de Suporte pode possuir uma versão ligeiramente desatualizada das informações de rastreamento enquanto o sistema realiza a sincronização após a falha de um Servidor Rastreador.

No caso da falha do Servidor Administrador líder, também pode ocorrer uma inconsistência temporária na relação entre pedidos e servidores de rastreamento. Como consequência, pedidos criados imediatamente antes da falha podem não ser recuperados, sendo necessário que o cliente realize uma nova solicitação após a conclusão do processo de eleição do novo líder.

Distribuição de Cópias

A distribuição das réplicas é realizada de forma dinâmica, permitindo que o sistema se adapte automaticamente à entrada ou saída de servidores.

Inicialmente, a responsabilidade pelo rastreamento de cada pedido é determinada por hashing consistente (Consistent Hashing). Cada Servidor Rastreador é representado em um anel lógico por nós virtuais, calculado a partir do identificador do servidor. Cada pedido também é posicionado nesse anel por meio de $\text{hash}(\text{idPedido})$, sendo associado ao primeiro Servidor Rastreador encontrado no sentido horário.

Os Servidores Administradores mantêm e replicam a lista de Servidores Rastreadores ativos, bem como a versão atual do anel de hashing. Quando um Servidor Rastreador falha, os nós virtuais associados a ele são removidos do anel. Apenas os pedidos pertencentes aos intervalos afetados são remapeados para os próximos servidores no anel. Quando um novo Servidor Rastreador entra no sistema, seus nós virtuais são adicionados ao anel e apenas parte dos pedidos dos servidores vizinhos precisa ser redistribuída.

Sempre que um Servidor Rastreador deixa de responder, os Servidores Administradores detectam a falha e utilizam a cópia armazenada pelo respectivo Servidor de Suporte para atualizar o anel de hashing consistente e redistribuir apenas os rastreamentos afetados entre os demais servidores disponíveis. Caso essa cópia de segurança não esteja acessível, o Servidor Administrador líder utiliza sua própria lista de mapeamento para reconstruir a distribuição dos pedidos, ainda que essa lista possa conter algumas informações temporariamente desatualizadas, como rastreamentos já encerrados.

Essa estratégia permite que o sistema realize o rebalanceamento automático da carga sempre que servidores entram ou saem da infraestrutura.

Protocolos de Consistência

O sistema utiliza uma abordagem híbrida para garantir a consistência das informações, combinando bibliotecas consolidadas para comunicação entre componentes e mecanismos implementados especificamente para a coordenação dos servidores.

A comunicação baseada em Publish-Subscribe será realizada por meio de um Message Broker, como Kafka, RabbitMQ, Google Pub/Sub ou AWS SNS/SQS. Essa infraestrutura é responsável por garantir a distribuição das mensagens entre produtores e consumidores, abstraindo os detalhes de roteamento e entrega.

Por outro lado, os mecanismos de coordenação entre os Servidores Administradores serão implementados especificamente para este projeto. A eleição de um novo líder após uma falha utilizará um algoritmo como o Bully Algorithm, enquanto a consistência das atualizações de localização será garantida pela comparação dos timestamps recebidos, descartando automaticamente mensagens antigas.

Para manter consistente a relação entre pedidos e Servidores Rastreadores, será empregado um protocolo de replicação baseado em líder (Leader-Based Replication). Nesse modelo, apenas o Servidor Administrador líder possui autoridade para modificar a tabela de mapeamento entre pedidos e servidores. Sempre que uma alteração é realizada, o líder propaga essa atualização para os demais Servidores Administradores e aguarda a confirmação da maioria antes de considerar a operação concluída. Essa estratégia reduz a probabilidade de divergências entre as réplicas e aumenta a confiabilidade do sistema.

Como o sistema possui no mínimo três Servidores Administradores, a maioria corresponde a dois ADMs. Assim, uma alteração no mapeamento pedido-servidor só é considerada confirmada quando o Servidor Administrador líder registra a alteração localmente e recebe confirmação de pelo menos mais um Servidor Administrador.

Ainda assim, caso o Servidor Administrador líder falhe durante o processo de propagação, pode haver inconsistências temporárias entre as listas mantidas pelos servidores restantes. Entretanto, como as alterações somente são efetivadas após a confirmação da maioria das réplicas, esse risco é significativamente reduzido, permitindo que o sistema retome seu funcionamento normalmente após a eleição de um novo líder.

Tolerância a falhas

Disponibilidade VS Confiabilidade

Neste projeto, o principal objetivo é garantir a disponibilidade do sistema. Como se trata de uma plataforma de entrega em tempo real, é preferível que o serviço permaneça

operacional mesmo durante falhas parciais, ainda que algumas informações possam ficar temporariamente inconsistentes. Dessa forma, clientes, entregadores e restaurantes continuam utilizando o sistema normalmente enquanto os mecanismos de recuperação restauram o estado consistente entre os servidores.

Consequências do teorema CAP

Considerando o Teorema CAP, o projeto prioriza Disponibilidade (Availability) e Tolerância a Partições (Partition Tolerance), adotando um modelo AP.

Isso significa que, mesmo durante falhas de comunicação entre servidores ou particionamentos da rede, o sistema continuará aceitando e processando solicitações. Como consequência, pode haver inconsistências temporárias entre as réplicas, especialmente durante a redistribuição de responsabilidades após falhas. Essas divergências são resolvidas posteriormente por meio dos mecanismos de sincronização e replicação, caracterizando um modelo de consistência eventual.

Falhas a serem toleradas

O sistema foi projetado para tolerar diferentes tipos de falhas distribuídas, buscando minimizar seu impacto sobre o funcionamento da aplicação.

As **falhas por omissão** ocorrem quando um servidor deixa de responder a uma requisição ou deixa de processar uma mensagem recebida. Nesses casos, após a detecção da falha por meio de timeout, a operação poderá ser reenviada ou redirecionada para outro servidor responsável.

As **falhas temporais** correspondem a atrasos na entrega das mensagens. Como as atualizações de localização são enviadas continuamente, cada mensagem contém um timestamp. Caso mensagens antigas sejam recebidas após mensagens mais recentes, elas são simplesmente descartadas, preservando apenas a informação mais atual do entregador.

As **falhas de resposta**, em que um servidor retorna uma resposta inválida ou inesperada, são tratadas por meio da repetição da operação ou do redirecionamento da requisição para outro servidor disponível.

As **falhas por crash** são tratadas através da replicação dos componentes. Quando um servidor deixa de funcionar completamente, outro servidor assume suas responsabilidades utilizando as informações previamente replicadas. Apenas um cenário de falha total, em que todos os servidores responsáveis por determinada funcionalidade estejam indisponíveis simultaneamente, não pode ser recuperado automaticamente.

Processos falhantes suportados

O sistema suporta falhas parciais de servidores e componentes individuais, desde que existam réplicas capazes de assumir suas responsabilidades. Assim, a

indisponibilidade de um ou mais Servidores Rastreadores, Servidores de Suporte ou Servidores Administradores não compromete completamente o funcionamento da aplicação.

Entretanto, falhas simultâneas que eliminem todas as réplicas responsáveis por uma determinada funcionalidade caracterizam um cenário de falha total e não podem ser recuperadas automaticamente.

Detecção de falhas

A detecção de falhas será baseada em mecanismos de heartbeat (keep-alive) e timeouts. Os servidores trocam periodicamente mensagens de verificação, por exemplo a cada 5 segundos, para confirmar sua disponibilidade. Caso um servidor deixe de responder dentro do tempo limite, por exemplo 15 segundos, ele será considerado indisponível pelo Servidor Administrador líder.

Após detectar a falha, o Servidor Administrador líder remove o servidor indisponível da lista de servidores ativos, atualiza o anel de hashing consistente e redistribui apenas os pedidos afetados pela saída desse servidor. Em seguida, a nova versão do mapeamento entre pedidos e Servidores Rastreadores é replicada para os demais Servidores Administradores.

Protocolos de tolerância a falhas

A tolerância a falhas do sistema é baseada na combinação de replicação de servidores, eleição de líderes e comunicação desacoplada por meio do modelo Publish-Subscribe.

O sistema é composto por três tipos principais de servidores:

- **Servidor Administrador (ADM):** mantém a relação entre pedidos e os Servidores Rastreadores responsáveis, coordena a redistribuição das cargas e executa o algoritmo de eleição do líder.
- **Servidor Rastreador (R):** responsável pelo acompanhamento em tempo real de um conjunto de pedidos.
- **Servidor de Suporte (SUP):** mantém uma cópia atualizada dos rastreamentos pertencentes a um Servidor Rastreador específico, permitindo sua recuperação em caso de falha.

Cada rastreamento é armazenado em pelo menos dois servidores, garantindo que a indisponibilidade de um deles não interrompa imediatamente o acompanhamento da entrega. Da mesma forma, existem pelo menos três Servidores Administradores, permitindo que um novo líder seja eleito e que alterações sejam confirmadas por maioria mesmo quando um dos administradores falhar.

Além disso, o uso de um Message Broker no modelo Publish-Subscribe reduz o acoplamento entre os componentes do sistema. Como produtores e consumidores não se

comunicam diretamente, falhas individuais afetam apenas os componentes envolvidos, sem interromper o funcionamento dos demais serviços.

Recuperação a partir da falha

A recuperação depende do tipo de falha identificado.

Em casos de falha por omissão ou falha de resposta, a operação é repetida ou redirecionada para outro servidor disponível após a detecção do timeout.

Nas falhas temporais, mensagens de localização recebidas fora de ordem são descartadas caso possuam timestamps inferiores ao da atualização mais recente armazenada.

Quando ocorre a falha de um Servidor Rastreador, seu Servidor de Suporte fornece a última cópia conhecida da lista de rastreamentos aos Servidores Administradores. A partir dessa informação, o líder atualiza o anel de hashing consistente e redistribui apenas os pedidos afetados pela falha entre os demais Servidores Rastreadores disponíveis. Caso a cópia do Servidor de Suporte não esteja disponível, o Servidor Administrador utiliza sua própria tabela de mapeamento para reconstruir a distribuição, ainda que algumas informações possam estar temporariamente desatualizadas.

Após a redistribuição dos pedidos, os Servidores Administradores atualizam a tabela de mapeamento entre pedidos e Servidores Rastreadores e propagam essa informação para os demais administradores. O novo Servidor Rastreador responsável passa então a receber as atualizações de localização publicadas pelo Message Broker, permitindo que clientes e restaurantes continuem acompanhando normalmente o andamento da entrega.

Se ocorrer a falha do Servidor Administrador líder, os demais administradores executam o algoritmo de eleição para selecionar um novo líder. Após a eleição, o novo coordenador utiliza as réplicas mantidas pelos administradores restantes para reconstruir o estado do sistema e retomar o gerenciamento das novas solicitações.